

All blogs / Databricks AI

AI RESEARCH

August 16, 2024

Beyond the Leaderboard: Unpacking Function Calling Evaluation

by [Kartik Sreenivasan](#), [Jeffrey Chen](#), [Pallavi Koppol](#), [Eitan Turok](#), [Bay Foley-Cox](#), [Asfandiyar Qureshi](#) and [Sam Havens](#)

1. Introduction

The research and engineering community at large have been continuously iterating upon Large Language Models (LLMs) in order to make them more knowledgeable, general-purpose, and capable of fitting into increasingly complex workflows. Over the last few years, LLMs have progressed from text-only models to having multi-modal capabilities; now, we are increasingly

[systems](#). This paradigm envisions an exposed to an end-to-end pipeline in and AI system model is more aligned

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

[Manage Preferences](#)[Reject All](#)[Accept All](#)

needs to have *tool use* capabilities. produce outputs to external *calling*, or the ability to interact this capability transforms LLMs more complex AI systems. However, interpret user requests accurately, correctly formatted function call

System: You are an AI Assistant who can use function calls to help answer the user's queries. You have access to several climate-related functions: `get_weather(city, state_abbr)`, `get_timezone(latitude, longitude)`, `get_nearest_station_id...`

User: What is the weather in San Francisco?

Given that the LLM has been made aware of several functions using the system prompt, it first needs to understand what the user wants. In this case, the question is fairly straightforward. Now, it needs to check if it needs external functions and if any of the available functions are relevant. In this case, the `get_weather()` function needs to be used. Even if the LLM has gotten this far, it now needs to plug in the correct arguments. In this case, it is clear that `city="San Francisco"` and `state_abbr="CA"`. Therefore, it needs to generate the following output:

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

this output to make the
to the user or feed it back into the

ry involving function calling requires
all LLMs possess this capability?
to measure it.

(BFCL) and the Nexus Function Calling Leaderboard (NFCL). We will discuss the specific aspects of function calling that these evals measure as well as their strengths and limitations. As we will see, it is sadly not a one-size-fits-all strategy. To get a holistic picture of a model's ability to perform function calling, we need to consider multiple factors and evaluation methods.

We'll share what we've learned from running these evaluations and discuss how it can help us choose the right model for certain tasks. We also outline strategies for improving an LLM's function calling and tool use abilities. In particular, we demonstrate that the performance of smaller, open source models like DBRX and LLama-3-70b can be elevated through a combination of careful prompting and parsing techniques, bringing them closer to or even surpassing GPT-4 quality in certain aspects.

What is function calling, and why is it useful?

Function calling is a tool that allows an LLM to interact with external systems using APIs and custom functions. Note that "tool use" and "function calling" are often used interchangeably in the literature; function calling was the first type of tool introduced and remains one of the most popularly used tools to date. In this blog, we refer to function calling as a specific type of tool use. In order to use function calling, the user first provides the model with a set of available

using JSON schemas. This gives the descriptions of each argument. When (any) functions are relevant. It then returns the necessary arguments.

examples that leverage function calling:

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

software infrastructure such as databases, e.g. via REST APIs.

Consider the following scenario: You are a large enterprise with a large amount of financial projection data in a SQL database. With function calling, you can now build a chatbot that has read access to that data. An internal user can ask the chatbot a complex question like "What is the projected revenue for our XYZ product in this fiscal year based on last year's trends?" and the chatbot can use function calling to request financial data between two dates which it can then use to give an accurate estimate. Here's an example of what this may look like:

User: "What is the projected revenue for our XYZ product in this fiscal year based on last year's trends?"

Chatbot: To answer this question, I'll need to retrieve some financial data. Let me check our database.

[Function Call: get_financial_data]

Parameters:

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

started on this date

e chatbot as a compound AI
the user who returns the

you with a projection for XYZ's

Projecting this trend forward, we can estimate that the total revenue for XYZ this fiscal year will be approximately \$11.5 million. This projection takes into account the current growth rate and the performance so far this year.

Would you like me to break down this calculation further or provide any additional information about the projection?

Batched Feature Extraction

Function calling usually refers to the LLM's ability to call a function from user-provided APIs or functions. But it also means the model must output the function call in the exact format defined by the function's signature and description. In practice, this is accomplished by using JSON as a representation of the function. This aspect can be exploited to solve a prevalent use case: extracting structured data in the form of JSON objects from unstructured data. We refer to this as "batched feature extraction," and find that it is fairly common for enterprises to leverage function calling in order to perform this task. For example, a legal firm could use an LLM with function-calling capabilities to process huge collections of contracts to extract key clauses, identify potential risks, and categorize each document based on its content. Using function calling, a large amount of data into simple JSONs

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

n natural language understanding
y expands the potential
estion of which LLM to use still
be extremely good at these tasks,
n failure modes rendering them
ngs. Therefore, like in all things LLM,

single-turn function calling and leverage the two most popular evals: Berkeley Function Calling Leaderboard (BFCL) and Nexus Function Calling Leaderboard (NFCL).

Berkeley Function Calling Leaderboard

The Berkeley Function Calling Leaderboard (BFCL) is a popular public function-calling eval that is kept up-to-date with the latest model releases. It is created and maintained by the creators of [Gorilla-openfunctions-v2](#), an OSS model built for function calling. Despite some limitations, BFCL is an excellent evaluation framework; a high ranking on its leaderboard generally indicates strong function-calling capabilities. As described in this [blog](#), the eval is composed of the following categories. (Note that BFCL also contains test cases with REST APIs and also functions in different languages. But the vast majority of tests are in Python which is the subset that we consider.)

1. **Simple Function** contains the simplest format: the user provides a single function description, and the query only requires that function to be called.
2. **Multiple Function** is slightly harder, given that the user provides 2-4 function descriptions and the model needs to select the best function among them to invoke in

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

calls in parallel with one user query. function description.

Simple and **Multiple**. The model is of them may need to be invoked

if none of the provided functions em.

fills it demands of the model:

arguments based on the query. correct function in addition to

calling and when not to. However, **Relevance Detection** only contains examples where none of the functions are relevant. Therefore, a model that is unable to ever perform function calling would seemingly score 100% on it. Nonetheless, given that a model performs well in the other categories, it becomes an extremely valuable eval. This once again underscores the importance of understanding these evals well and viewing them holistically.

Each of the above categories can be evaluated by checking the Abstract Syntax Tree (AST) or actually executing the function call. The AST evaluation first constructs the abstract syntax tree of the function call, then extracts the arguments and checks if they match the ground truth's possible answers. (Footnote: For more details refer to: https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html#bfcl)

We found that the AST evaluation accuracy correlates well with the Executable evaluation and, therefore, only considered AST.

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

lies bespoke parsing for several models
e fairly across models (Note: in our
parsing across models to only include
out.)

oo easy and not representative of real-
mple and multiple appear to be
t of the best models have already

it an effective eval for enterprise function calling. It is from the creators of the NexusRaven-v2 which is an OSS model aimed at function calling. While the NFCL reports that it outperforms even GPT-4, it only gets 68.06% on BFCL. This discrepancy once again reveals the importance of understanding what the eval numbers on a particular benchmark mean for a specific application.

The NFCL categories are split based on the source of their APIs rather than the kind of evaluation. However, they also differ in difficulty, as we describe below.

1. **NVD Library:** The queries in this category are based on the two search APIs from the National Vulnerability Database: searchCVE and searchCPE. Since there are only two APIs to choose from, this is a relatively easy task that only requires calling one of them. The complexity arises from the fact that each function has around 30 arguments.
2. **VirusTotal:** These are based on the VirusTotal APIs which are used to analyze suspicious files and URLs. There are 12 APIs but they are simpler than NVD. Therefore, models typically score slightly higher on VirusTotal than NVD. VirusTotal still requires only a single function call.
3. **OTX:** These are based on the Open Threat Exchange APIs. There are 9 extremely simple APIs and this is usually the category where most models score the highest.

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

ed to querying details about the questions require nested ; it tricky for most models. While a ny require nesting of up to 7

PIs used to retrieve climate data. en require multiple parallel calls and most models.

he VirusTotal benchmark, but the red. This is one of the hardest designed to output nested function

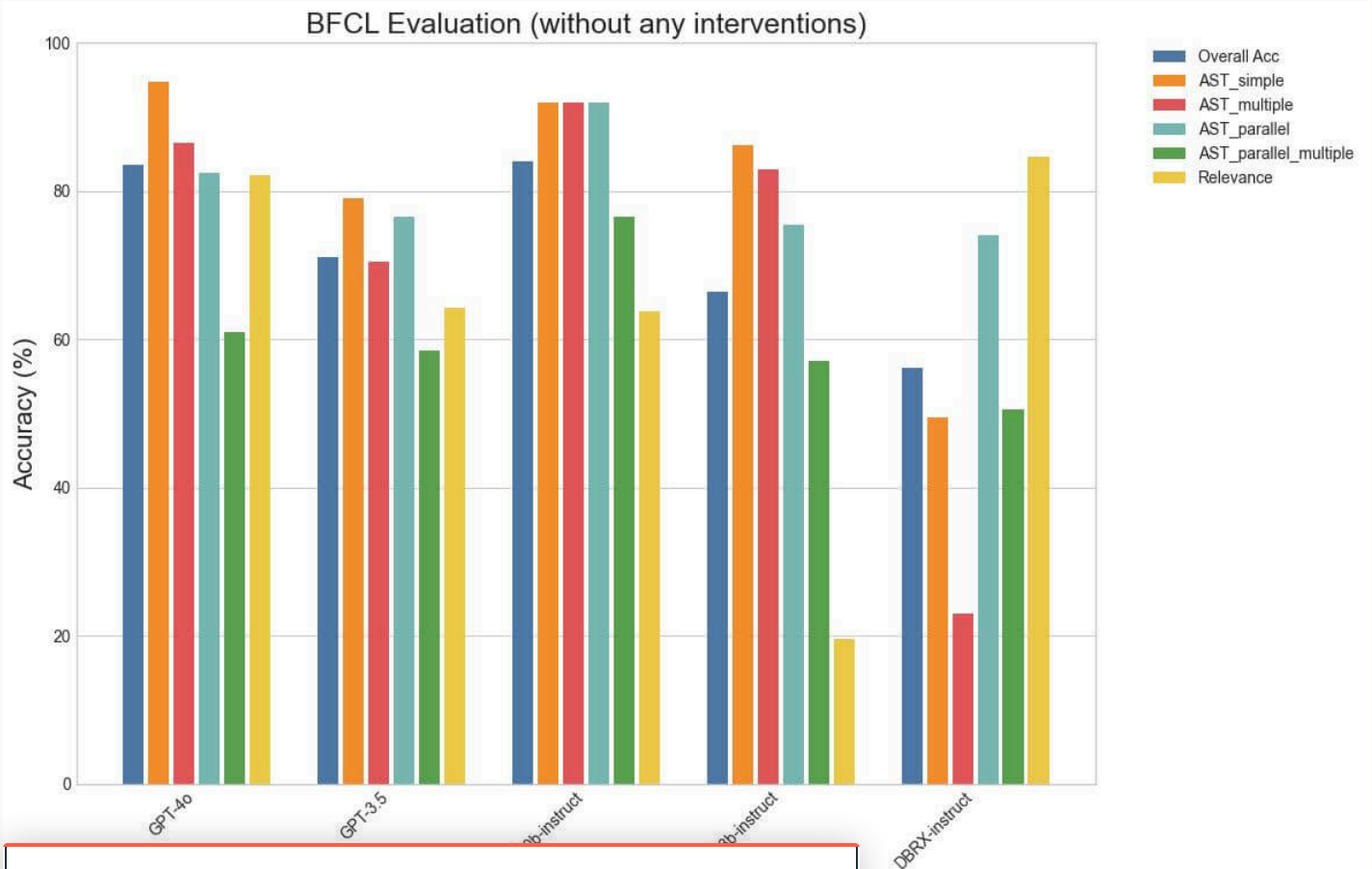
D benchmark, but the questions he models we have tested were

the BFCL taxonomy, **NVD**, **VirustTotal** and **OTX** categories would be labeled as **Multiple Function** but with more candidate functions to choose from. The parallel examples in **Climate** would be categorized as **Parallel Function**, while the nested examples in the remaining categories do not have an equivalent. In fact, nested function calls are a somewhat unusual eval since they are typically handled through multi-turn interactions in the function-calling world. This also explains why most models, including GPT-4, struggle with them. In addition to likely being out of distribution from the model's training data, the LLM must plan the order of function invocations and plug them into the correct argument of the later function calls. We find that despite not being representative of typical use cases, it is a useful eval since it tests both planning and structured output generation while being less susceptible to eval overfitting.

Scoring for NFCL is based purely on string matching on the final function call generated by the model. While this is not ideal, we find that it rarely, if at all, leads to false positives.

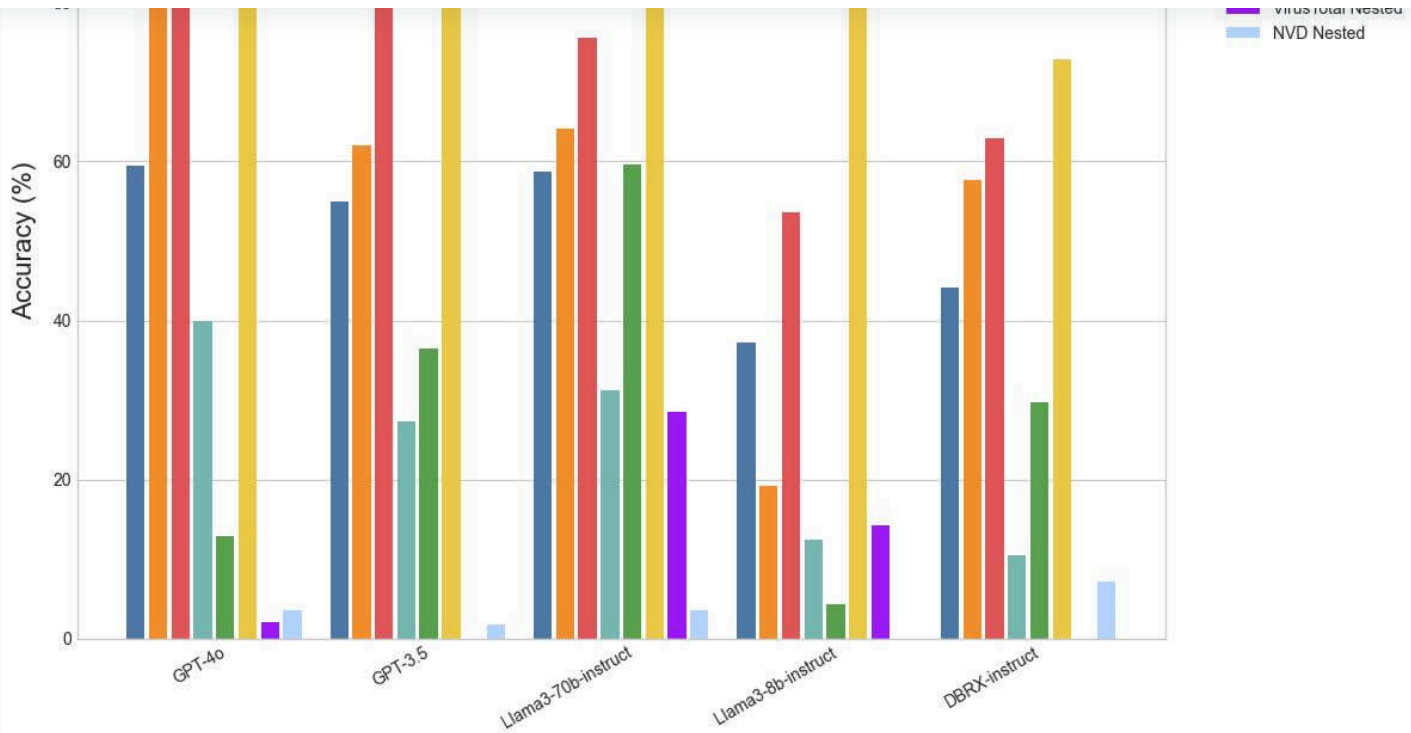
Strengths	Weaknesses
Other than OTX , none of the categories appear to be	Most function-calling implementations refer to the OpenAI spec; therefore, they are ly to solve the nested categories ut breaking it down into a multi-turn ction.
We Care About Your Privacy Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our Cookie Notice. To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."	Scoring is based on exact string ning of the function calls and may be g to false negatives.
	of the function descriptions are lacking an be improved. Furthermore, several of are atypical in that they have a large er of arguments or have no required nents.
	of the examples test relevance ction.

prompting and parsing uniform across models.



We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."



We found that evaluating even on publicly available benchmarks is sometimes nuanced as the behavior can vary wildly with different generation kwargs. For example, we find that accuracy can vary as much as 10% in some categories of BFCL when generating with Temperature 0.0 vs Temperature 0.7. Since function-calling is a fairly programmatic task, we find that using

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

loss models. We made the decision system prompt as repeating them in multi-turn conversations. We also implementations for both NFCL and BFCL. er than that from the publicly are higher. This is because the ng for DBRX.

with the expected ordering, in that ama3-70b-instruct, then GPT-3.5 to GPT-4o on **Climate** and urprisingly, DBRX-instruct performs for function-calling. We suspect that this is because it is not biased against nested function calls and simply solves it as a

clearly inferior model. We posit that a high score on BFCL is a necessary, rather than sufficient, condition to be good at function calling. Low scores indicate that a model clearly struggles with function calling whereas a high score doesn't guarantee that a model is better at function calling.

4. Improving Function-calling Performance

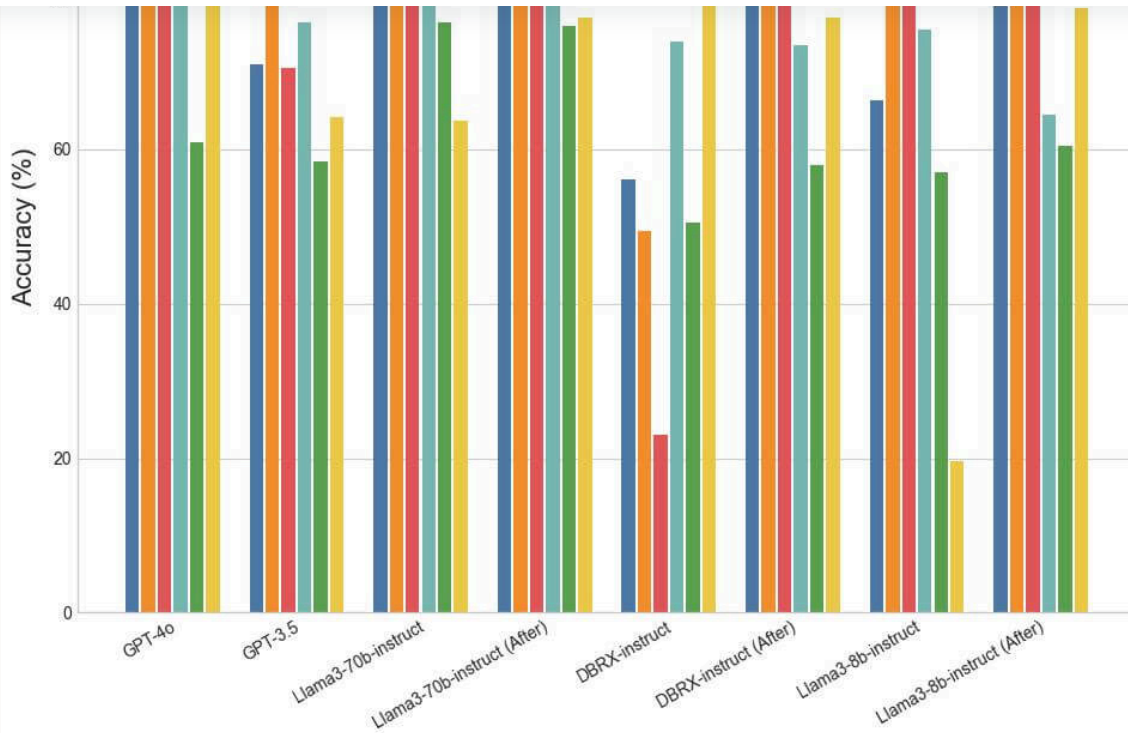
Once we have a reliable way to evaluate a capability and know how to interpret the results, the obvious next step is to try and improve those results. We found that one of the keys to unlocking a model's function-calling abilities is specifying a detailed system prompt that gives the model the ability to reason before making a decision on which function to call, if any. Further, directing it to structure its outputs using XML tags and a somewhat strict format makes parsing the function call easy and reliable. This eliminates the need for bespoke parsing methods for different models and applications.

Another key element is ensuring that the model is given access to the details of the function, its arguments and their data types in an effective format. Ensuring that each argument has a data type and a clear description helps elevate performance. Few-shot examples of expected model

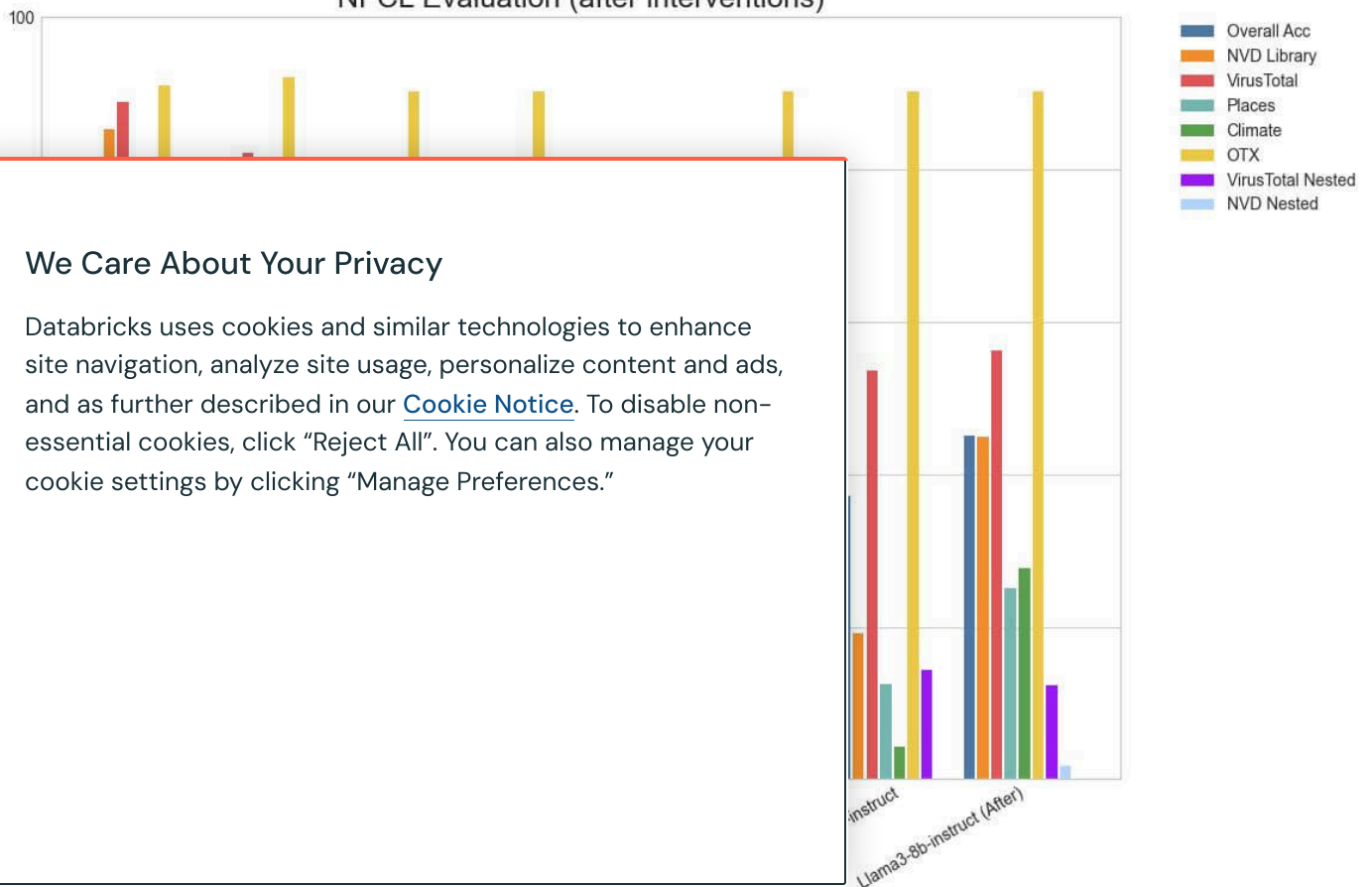
evaluate the relevance of the passed functions. In our prompt, we used few-provided functions one-by-one deciding which function to call.

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."



NFCL Evaluation (after interventions)



We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

from 84.58% to 77.08%. The reason for this is due to a limitation in the way relevance detection is implemented. Since all the test cases only contain irrelevant functions, a model that is poor at function-calling and calls functions incorrectly or even fails to ever call a function will do exceptionally well in this category. Therefore, it can be misleading to view this number outside of the context of its overall performance. The high relevance detection accuracy of DBRX-instruct before our changes is because its outputs were often structurally flawed and therefore its overall function-calling performance was poor.

The general directions in our system prompt look like this:

Please use your own judgment as to whether or not you should call a function. In particular, you must follow these guiding principles:

1. You may assume the user has implemented the function themselves.
2. You may assume the user will call the function on their own. You should NOT ask the user to call the function and let you know the result; they will do this on their own. You just need to pass the name and arguments.

3. Never call a function twice with the same arguments. Do not repeat your function

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

question, DO NOT MAKE any

: listed in this prompt, no
interpreter either. DO NOT MAKE

formatting rules:

and within {tool_list_start}
that are not listed between

ms to the schema of the
start}[...list of tool calls...]

VALID JSON BETWEEN THE {tool_call_start} AND {tool_call_end} TAGS!

Rule 3: If user decides to run the function, they will output the result of the function call in the following query. If it answers the user's question, you should incorporate the output of the function in your following message.

We also specified that the model uses the `<thinking>` tag to generate the rationale for the function call while specifying the final function call within `<tool_call>` tags.

Supposed the functions available to you are:

`<tools>`

[{'type': 'function', 'function': {'name': 'determine_body_mass_index', 'description': 'Calculate body mass index given weight and height.', 'parameters': {'type': 'object', 'properties': {'weight': {'type': 'number', 'description': 'Weight of the individual in kilograms. This is a float type value.', 'format': 'float'}, 'height': {'type': 'number', 'description': 'Height of the individual in meters. This is a float type value.', 'format':

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

tion': 'Compute the product
properties': {'numbers': {'type':
numbers to be added up.'},
ber of decimal places to

_calculate', 'description':
linates.', 'parameters': {'type':
: {'type': 'number'},
ongitude.'}, 'coordinate_2':
second coordinate, a pair of
dinate_2']}]}

Let's start with a list of functions I have access to:

- `determine_body_mass_index`: since this function is not relevant to getting the current time, I will not call it.
- `math_prod`: since this function is not relevant to getting the current time, I will not call it.
- `distance_calculator_calculate`: since this function is not relevant to getting the current time, I will not call it.

None of the available functions, [`determine_body_mass_index`, `math_prod`, `distance_calculator`] are pertinent to the given query. Please check if you left out any relevant functions.

As a Large Language Model, without access to the appropriate tools, I am unable to provide the current time in New York.

</thinking>

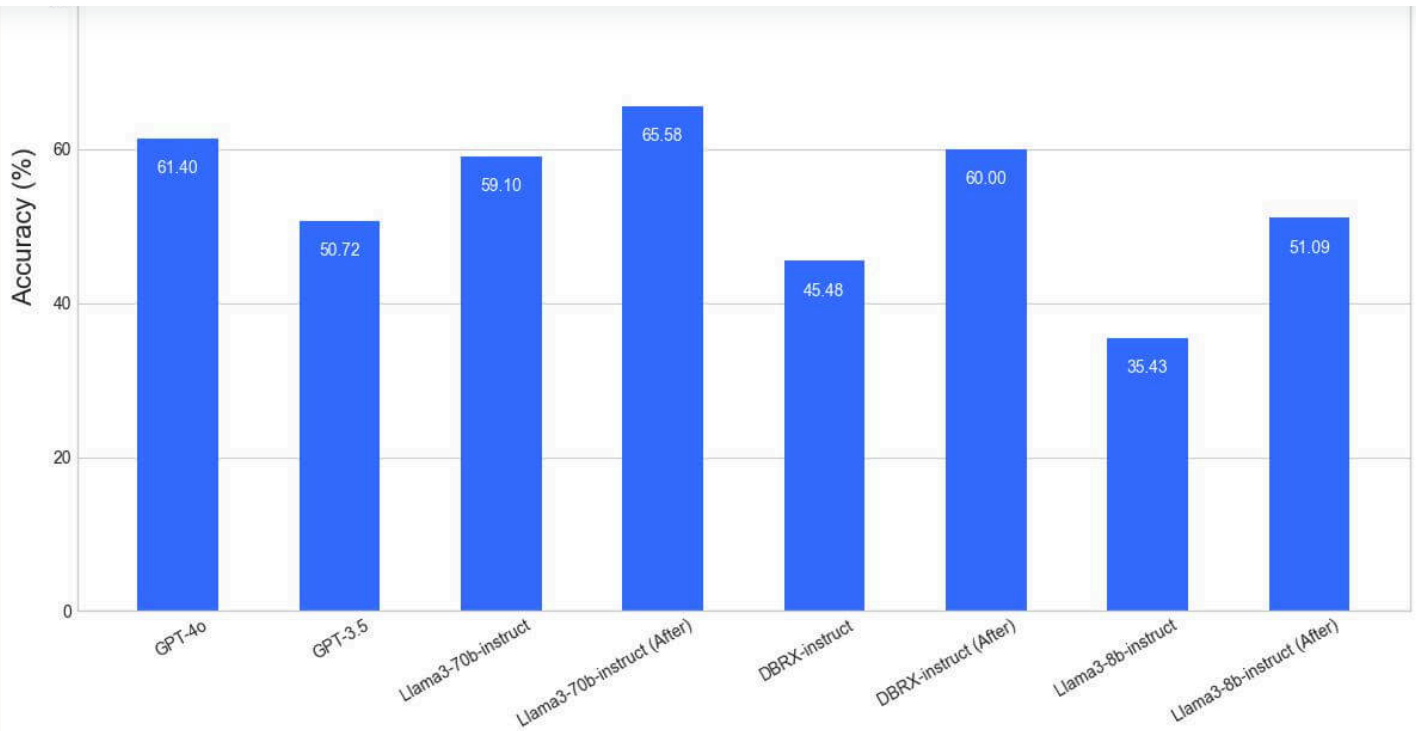
While the exact system prompt that we used may not be suitable for all applications and all models, the guiding principles can be used to tailor it for specific use cases. For example, with Llama-3-70b-instruct we used an abridged version of our full system prompt which skipped the few-shot examples and omitted some of the more verbose instructions. We would also like

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

h and we encourage using

subcategories in BFCL and NFCL
ignored the Climate column, since
y, we upweighted relevance
ty of models to perform function



The aggregate metric shows that Llama3-70b-instruct, which was already approaching GPT-4o in quality, surpasses it with our modifications. Both DBRX-instruct and Llama3-8b-instruct which start at below GPT-3.5 quality surpass it and begin to approach GPT-4o quality on these benchmarks.

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

Whether they can generate output results above, the best open source they are still susceptible to these shortcomings is by using (coding), a decoding technique that tokens. This is done by modifying s that would violate given structural ries are [Outlines](#), [Guidance](#), and on gives strong guarantees that are rent implementation of function nly presented results with emphasize that a well-implemented structured generation pipeline should further improve the function-calling abilities of an LLM.

Here are some key takeaways:

1. **Comprehensive evaluation:** No single benchmark tells the whole story. A holistic approach, combining multiple evaluation frameworks like BFCL and NFCL is crucial to understanding a model's function calling capabilities.
2. **Nuanced interpretation:** High scores on certain benchmarks, while necessary, are not always sufficient to guarantee superior function-calling performance in practice. It is essential to understand the strengths and limitations of each evaluation metric.
3. **The power of prompting:** We have demonstrated that careful prompting and output structuring can dramatically improve a model's function-calling abilities. This approach allowed us to elevate the performance of models like DBRX and Llama-3, bringing them closer to or even surpassing GPT-4o in certain aspects.
4. **Relevance detection:** This often-overlooked aspect of function calling is crucial for real-world applications. Our improvements in this area highlight the importance of guiding models to reason about function relevance.

To learn more about function calling, review our [official documentation](#) and try out our [Foundational Model APIs](#).

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

your inbox

delivered to your inbox.



Databricks processing my personal data in accordance with its [Privacy Policy](#).

[View all blogs](#)

Why Databricks 

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."

Databricks Inc.
160 Spear Street, 15th Floor
San Francisco, CA 94105
1-866-330-0121

See Careers
at Databricks



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

[Privacy Notice](#) | [Terms of Use](#) | [Modern Slavery Statement](#) | [California Privacy](#)

We Care About Your Privacy

Databricks uses cookies and similar technologies to enhance site navigation, analyze site usage, personalize content and ads, and as further described in our [Cookie Notice](#). To disable non-essential cookies, click "Reject All". You can also manage your cookie settings by clicking "Manage Preferences."